

KryptoVault

Technical Project Summary

1. Project Overview

Problem

Centralized document and identity platforms create single points of failure. A database breach can expose many users at once, while document owners have limited control once sensitive files are shared.

Solution

KryptoVault combines client-side encryption, wallet-based authentication, smart-contract access control and off-chain encrypted storage. Files are encrypted before leaving the user's device, while blockchain records ownership, permissions, hashes, versions and audit events.

Target Users

- Government departments handling sensitive citizen or internal documents.
- Universities and institutions managing credentials and controlled records.
- Enterprises and compliance teams requiring traceable access and revocation.
- Individuals who need secure, owner-controlled sharing of important files.

Why Blockchain Is Actually Needed

Blockchain is not used to store the actual files. Its purpose is to provide a tamper-resistant shared record of ownership, access grants, revocations, file hashes and version history. This removes the need to trust a single database administrator for the integrity of the access log.

2. System Architecture

Frontend

React + Vite frontend with Tailwind CSS. It handles the user interface, file selection, local encryption/decryption, hashing and wallet interaction.

Backend

Node.js + TypeScript + Express backend. It verifies requests, coordinates access checks, communicates with MongoDB and serves encrypted payloads and wrapped keys to authorized users.

Database

MongoDB stores user metadata, wallet addresses, public encryption keys, asset metadata, access records, wrapped AES keys and version metadata. Plaintext files and raw AES keys are not stored.

Object / File Storage

Encrypted file blobs are stored off-chain using MongoDB GridFS or another object-storage layer. Keeping large encrypted files off-chain avoids unnecessary blockchain cost and storage limits.

Blockchain

A Solidity smart contract on an EVM network such as Ethereum Sepolia stores asset ownership, current file hashes, permission states, validity periods, version information and immutable audit events.

KYC Layer

The MVP can associate a wallet with profile and KYC-status metadata. Raw KYC documents should remain off-chain. Verifiable Credentials and selective-disclosure proofs are future-scope features, not required for the present prototype.

Clean Architecture Diagram



3. Security Flow

- 1. File Upload:** The owner selects a file. The frontend generates a random 256-bit AES key.
- 2. AES Encryption:** The file is encrypted locally with AES-256-GCM before transmission.
- 3. SHA-256 Hashing:** A SHA-256 fingerprint of the encrypted payload is computed and later stored on-chain.
- 4. Wallet Authentication:** The user proves wallet ownership using a MetaMask signature. MetaMask is used for identity and blockchain signing.
- 5. Public / Private Encryption Keys:** The AES file key is wrapped using the recipient's public encryption key. The recipient's private encryption key is used locally to unwrap it. This key pair is separate from MetaMask wallet keys.
- 6. Smart-contract Authorization:** The backend checks the smart contract for READ or WRITE permission and verifies validity periods before releasing encrypted data.
- 7. File Access:** If authorized, the backend returns the encrypted file and recipient-specific wrapped AES key. Decryption occurs locally on the recipient's device.
- 8. Revocation:** Weak revocation removes permission and blocks future delivery. Strong revocation additionally generates a new AES key, re-encrypts the current file and re-wraps the new key for remaining users.
- 9. Version Update:** After a strong revocation or authorized edit, the new ciphertext hash and version information are committed to the blockchain.

Important limitation: strong revocation prevents a revoked user from decrypting newly re-encrypted or future asset states. It cannot erase plaintext or old copies that the user already legitimately obtained.

4. On-chain vs Off-chain

Stored On-chain

- Asset identifier and owner wallet.
- Current SHA-256 file hash and version number.
- READ / WRITE / NONE permission state.
- validFrom and validUntil access periods.
- Grant and revoke audit events.
- Version-commit events and related reason or message.

Stored Off-chain

- AES-256-GCM encrypted file blobs.
- Recipient-specific wrapped AES keys.
- Public encryption keys.
- User profile and KYC-status metadata.
- Storage paths and file metadata.
- Encrypted version data, IVs and authentication tags.

What Is Never Intentionally Stored on the Server

- Plaintext files.
- Raw AES file keys.
- Wallet private keys.
- User private encryption keys.
- Raw passwords.

Why Files Are Not Stored Directly on Blockchain

Public blockchains are inefficient and expensive for large binary files. KryptoVault therefore uses a hybrid architecture: encrypted files remain off-chain, while blockchain stores only compact information that benefits from immutability and independent verification.

What the Smart Contract Stores

The smart contract maintains ownership, hashes, version state, permission records, access validity windows and lifecycle events such as registration, access grant, revocation and version commit.

5. Prototype / Implementation

Current Frontend Features

- React-based user interface for the KryptoVault workflow.
- Wallet connection and MetaMask-oriented authentication flow.
- File upload and access-management screens.
- Client-side encryption/decryption architecture.
- Grant, revoke and versioning controls represented in the application flow.

Current Backend Status

The backend uses Node.js, TypeScript and Express with MongoDB integration. API structure, validation and storage flow are being integrated. Production hardening, complete blockchain wiring and full end-to-end testing should be treated as ongoing implementation work.

MetaMask

MetaMask is used for wallet connection, signatures and blockchain transaction authorization. The application must never read or store the MetaMask private key.

Mock vs Real Components

The current prototype may use mock or fallback behavior where live blockchain or backend integration is incomplete. In the final demo, the team should clearly identify which actions are live on Sepolia and which are prototype simulations.

Tech Stack

- Frontend: React, Vite, Tailwind CSS.
- Backend: Node.js, TypeScript, Express, Zod.
- Database: MongoDB / MongoDB Atlas, Mongoose.
- Blockchain: Solidity, Hardhat, Ethereum Sepolia.
- Web3: Ethers.js, MetaMask.
- Cryptography: AES-256-GCM, SHA-256, asymmetric key wrapping.
- Development: Git, GitHub, VS Code.

6. Feasibility, Security & Future Scope

Database Compromise Scenario

If MongoDB or file storage is breached, an attacker should obtain encrypted file blobs, wrapped keys and metadata rather than plaintext files or raw AES keys. On-chain SHA-256 hashes also allow tampering with stored ciphertext to be detected.

Private-Key Compromise Limitation

If a user's private encryption key is stolen, file keys wrapped for that user may be exposed. Strong revocation can protect new/current versions by rotating the AES key, but no system can remotely erase plaintext already copied by an authorized user.

Weak vs Strong Revocation

Weak revocation is faster and cheaper: the smart contract sets permission to NONE and the backend stops future delivery. Strong revocation also rotates the AES key, re-encrypts the current file, re-wraps the new key for remaining authorized users and commits the new hash/version.

Scalability

The hybrid model scales because large files stay off-chain while only hashes, permissions and events are stored on-chain. For very large files, chunked encryption and background workers can reduce browser memory pressure.

Future Production Upgrades

- IPFS or production-grade object-storage abstraction for encrypted files.
- Chunked encryption and Web Workers for multi-gigabyte assets.
- Hardware-backed or WebAuthn protection for encryption keys.
- Folder, group and role-based permission management.
- Verifiable Credentials and privacy-preserving selective disclosure.
- Smart-contract audits, monitoring, stronger recovery processes and production security hardening.

KryptoVault's present design is technically feasible because it relies on established web, cryptographic and EVM technologies rather than experimental primitives.